

TalentLink Job Portal

A PROJECT REPORT

Submitted by

24BCS11025 - Suraj

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



Chandigarh University

APRIL 2026

Section 1

Overview

1.1 Project Vision

TalentLink is a comprehensive job management platform designed to modernize the recruitment lifecycle. While traditional job boards act as simple data repositories, TalentLink functions as an "Intelligent Assistant." By merging the high-speed reactivity of a **React 19** frontend with the enterprise-grade stability of a **Spring Boot 3** backend, the project provides a professional environment for managing corporate vacancies.

1.2 The Problem Statement

Recruiters often struggle with:

1. **Content Quality:** Shorthand notes leading to unprofessional or vague job postings.
2. **Market Awareness:** A lack of immediate context regarding salary benchmarks and talent demand in specific geographic locations.
3. **Data Persistence:** Managing volatile data that needs to survive server restarts and scale for enterprise use.

1.3 Objectives

The primary objective was to build a production-ready application that demonstrates:

- **AI Augmentation:** Using Generative AI to automate content creation.
- **Persistent Storage:** Transitioning from in-memory testing to **MySQL** production databases.
- **Decoupled Communication:** Implementing a robust REST API bridge between different tech stacks.

Section 2

Features (Functionality)

2.1 Content Refinement

The standout feature of TalentLink is its integration with the **Google Gemini 2.5 API**. Within the job creation form, the "Optimize" engine acts as a professional editor. It uses advanced prompt engineering to take raw, unformatted text and convert it into a structured, high-impact job description. This ensures that every posting published to the database meets corporate standards.

2.2 Real-Time Market Intelligence

For every active job posting, the system offers a "Get Insights" feature. Instead of the recruiter manually researching industry trends, the AI analyzes the `Title` and `Location` fields to return a concise summary of:

- **Salary Brackets:** Estimated compensation for the role.
- **Demand Analysis:** Current market saturation for that specific skill set.

2.3 Full-Stack CRUD Operations

The application implements the complete **Create, Read, Update, and Delete** lifecycle:

- **Create:** Dynamic forms capture job details and send them as JSON to the Java backend.
- **Read:** An asynchronous fetching system loads existing jobs into the UI upon application startup.
- **Delete:** A secure deletion endpoint allows recruiters to remove filled or expired positions from the permanent MySQL storage.

2.4 Professional UI/UX

The design utilizes a **Monochromatic Slate & Blue** aesthetic. By leveraging the latest Tailwind CSS v4, the project achieves a high-performance, responsive layout. Features include custom-designed cards, interactive AI-loading states, and a mobile-responsive grid system.

Section 3

Flow of Project

The workflow of TalentLink is designed to be seamless, moving from user interaction to permanent storage in a matter of milliseconds.

3.1 The Creation & Optimization Flow

1. **Input:** The user enters the initial job details into the React form.
2. **AI Transformation:** If the user clicks "Optimize," a `POST` request is sent to the Gemini API. The AI returns a polished description which React updates in the local state.
3. **Submission:** When "Publish" is clicked, React sends a `POST` request containing the job object to `http://localhost:8080/api/jobs`.

3.2 The Backend Processing Flow

1. **Reception:** The `JobController` in Spring Boot receives the JSON.
2. **Mapping:** The **Jackson** library maps the JSON fields into the `Job` Java Entity using standard Getters and Setters.
3. **Persistence:** The `JobRepository` calls the `save` method, which Hibernate translates into a `SQL INSERT` statement for the MySQL server.

3.3 The Retrieval & UI Sync Flow

1. **Lifecycle Trigger:** When the React app loads, the `useEffect` hook triggers a `GET` request.
2. **Data Fetch:** Spring Boot queries the MySQL database for all records and returns them as a JSON array.
3. **State Update:** React receives the array and updates the `postings` state, causing the browser to render the active job cards instantly.